SOFTWARE REQUIREMENT SPECIFICATION

FOR

AIRLINE MANAGEMENT SYSTEM

1. INTRODUCTION

1.1 Purpose

This document provides a detailed description of the requirements for the **Airline Management System (AMS)**.

The purpose of this system is to digitally manage and automate airline operations, including:

- Flight scheduling and management
- Passenger booking and reservation
- Ticket generation and cancellation
- Staff and crew management
- Check-in and boarding processes
- Payment processing
- Report generation

This SRS is intended for:

- Project developers
- Airline management authorities
- System administrators
- Testing and maintenance teams

1.2 Scope

The system is designed to manage airline operations in an efficient and centralized manner. It supports three main user roles: **Admin, Staff, and Passenger**.

The system provides role-based dashboards and enables:

- Flight management and scheduling
- Passenger booking and ticketing
- Payment and cancellation handling

- Check-in and boarding operations
- Staff and operational management

The system will:

- Allow administrators to manage flights, schedules, staff, and system configurations
- Enable staff to handle passenger check-ins, update flight status, and assist operations
- Allow passengers to search flights, book tickets, cancel bookings, and view travel details

---

1.3 Definitions, Acronyms, Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
AMS	Airline Management System
Admin	System Administrator – manages entire system
Staff	Airline staff responsible for operations
Passenger	User who books and travels
PNR	Passenger Name Record
UI	User Interface
DB	Database
API	Application Programming Interface
HTML/CSS/JS	Web technologies used for frontend
MySQL	Relational database management system
MVC	Model-View-Controller architecture

1.4 References

- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications
- Airline Management System Project Documentation, 2026
- Use Case Specifications Document – AMS
- Standard Airline Operational Procedures

1.5 Overall Description (Overview)

The system follows a **Three-Tier Architecture**:

1. Presentation Layer – User dashboards (Admin, Staff, Passenger)
2. Application Layer – Business logic (booking, scheduling, payment processing)
3. Database Layer – MySQL

Technology Stack:

- Frontend: HTML, CSS, JavaScript
- Backend: PHP
- Database: MySQL
- Tools: VS Code, GitHub, XAMPP Server

The remaining sections of this SRS are organized as follows:

- **Section 2 – Overall Description:** Describes product perspective, functions, user characteristics, constraints, assumptions, and dependencies
- **Section 3 – Specific Requirements:** Includes functional requirements, use cases, interface requirements, performance requirements, and system attributes
- **Section 4 – Supporting Information:** Includes appendices and additional details

2. GENERAL DESCRIPTION

2.1 Product Perspective

The Airline Management System (AMS) is a new, self-contained web-based platform. It does not replace any existing legacy system but is built from the ground up to digitally manage airline operations such as flight scheduling, booking, and passenger handling.

The system follows a three-tier architecture:

Presentation Layer:
Role-based dashboards for Admin, Staff, and Passenger, built using HTML, CSS, JavaScript, and Bootstrap.

Application Layer:
Server-side business logic handling flight scheduling, booking processes, payment handling, check-in operations, and role-based access control, built using Java/PHP/Node.js with Express.js.

Database Layer:
Persistent storage of all flight, passenger, booking, and staff data using MySQL/PostgreSQL.

The system communicates via standard HTTP/HTTPS protocols. All user interactions happen through a web browser without any proprietary plugins.

2.2 Product Functions

Major functional modules:

User Role Functionalities / Product Functions

Admin Module

- Add and manage flights
- Manage staff accounts
- Monitor bookings
- Generate reports
- Manage system settings

Staff Module

- Manage passenger details
- Perform check-in operations
- Update flight status (delay/cancel/on-time)
- Assist boarding process

Passenger Module

- Search flights
- Book tickets
- Make payments
- Cancel bookings
- View ticket and booking history

2.3 User Characteristics

Admin:
Technically skilled personnel responsible for managing the entire system. Expected to have knowledge of system configuration and airline operations.

Staff:
Airline employees responsible for operational activities such as check-in, boarding, and passenger support. Requires basic computer knowledge.

Passenger:
General users who book and travel using the system. Expected to have basic internet browsing skills.

2.4 User Constraints

- All users must have a registered account with valid credentials before accessing the system.
- Role-based access control strictly limits each user's actions to their designated module.
- Passengers can only access their own bookings and travel details.
- Staff can only perform operations assigned to them.
- Password policy must be enforced; accounts are locked after a maximum of 3 failed login attempts.
- Ticket booking depends on seat availability.
- Payment must be completed successfully before ticket confirmation.

2.5 Assumptions and Dependencies

- Users have reliable internet access when interacting with the system.
- The hosting server (WAMP Server or equivalent) is operational and accessible.
- The MySQL/PostgreSQL database server is running and properly configured.
- Flight schedules are updated regularly by the admin.
- Payment gateway integration is available and functioning properly.
- Staff accounts are created before assigning operational tasks.
- Passengers provide valid personal and identification details during booking.

2.6 Apportioning of Requirements

The following requirements are identified as deferred to future versions of the system:

- Mobile application (Android/iOS) for passengers and staff
- Integration with third-party payment wallets and international gateways
- Automated email/SMS notifications for booking confirmation and flight updates
- Advanced analytics and reporting dashboard for airline performance
- Real-time flight tracking system
- Integration with airport management systems

3. SPECIFIC REQUIREMENTS

3.1 Interface Requirements

3.1.1 External Interface

The system provides a web-based user interface accessible via standard browsers (Chrome, Firefox, Edge). All user interactions are performed through the browser. The external interface includes:

- Login page – accessible to all user types
- Role-specific dashboards – displayed upon successful authentication
- Forms for data entry (flight creation, booking, check-in, payment)
- Ticket display and download interface
- Booking history and cancellation interface

3.1.2 Hardware Interface

The system has no direct hardware interface requirements. It runs on standard web-capable devices (laptops, desktops, or tablets). Minimum requirements are:

- A device with an updated web browser
- Internet connectivity (minimum 1 Mbps recommended)
- Server hardware capable of running WAMP/LAMP stack with MySQL/PostgreSQL

3.1.3 Software Interface

- Frontend: HTML5, CSS3, JavaScript, Bootstrap
- Backend: Java / PHP / Node.js with Express.js
- Database: MySQL or PostgreSQL
- Development Tools: VS Code, Git & GitHub, WAMP Server
- Operating System: Windows / Linux

3.1.4 Communication Interface

- HTTP/HTTPS protocol for all client-server communication
- Internet/LAN connectivity
- Database communication via SQL

3.2 Functional Requirements

3.2.1 Use Case Model / Information Flow

This section presents the Use Case Model for the Airline Management System.

Actors

Actor	Description
Admin	Manages flights, staff, system settings
Staff	Handles check-in, flight updates
Passenger	Books tickets and travels

Use Case Relationships

- **<<include>>** → Mandatory actions (e.g., Booking includes Payment)
- **<<extend>>** → Optional actions (e.g., Booking extends to Cancellation)

3.2.2 Use Case Specifications

UC-001: Login to System (Common)

- **Actors:** Admin, Staff, Passenger
- **Description:** User logs into system
- **Preconditions:** User registered
- **Postconditions:** Dashboard displayed
- **Normal Flow:** Enter credentials → Login → Redirect
- **Alternate Flow:** Invalid credentials
- **Exceptions:** Account locked / Server error
- **Priority:** High

ACTOR – ADMIN

UC-01: Add Flight

- Description: Admin creates flight
- Preconditions: Admin logged in
- Postconditions: Flight stored
- Normal Flow: Enter details → Save
- Alternate Flow: Invalid data

- Exceptions: Database error
- Priority: High

UC-02: Manage Staff

- Add/Edit/Delete staff
- Staff records updated

UC-03: View Reports

- View booking reports
- View revenue reports

UC-04: Manage Flight Schedule

- Update timings
- Cancel flights

ACTOR – STAFF

UC-05: Manage Passengers

- View passenger list
- Update passenger info

UC-06: Check-in Passenger

- Verify ticket
- Assign seat
- Generate boarding pass

UC-07: Update Flight Status

- Delay / Cancel / On-time

ACTOR – PASSENGER

UC-08: Search Flights

- Enter source & destination
- View available flights

UC-09: Book Ticket

- Select flight
- Enter details
- Make payment

UC-10: Cancel Ticket

- Cancel booking
- Refund processed

UC-11: View Booking

- View ticket history

3.3 Performance Requirements

1. System supports at least **500 concurrent users**
2. Login response within **3 seconds**
3. Booking confirmation within **5 seconds**
4. Dashboard loads within **3–5 seconds**
5. Payment processing within **5 seconds**
6. System availability **99% uptime**

3.4 Logical Database Requirements

The system uses a relational database (MySQL/PostgreSQL).

Main Tables

Table	Description
users	Stores user details
flights	Flight information

Table	Description
bookings	Ticket bookings
passengers	Passenger data
payments	Payment details

Data Integrity Constraints

- Flight ID, User ID must be unique
- Seat availability must be validated
- Foreign key relationships maintained
- Payment must be valid before booking confirmation

Database Design Approach

- Relational Model
- Normalization (3NF)
- Primary & Foreign Keys
- Role-Based Access Control

3.5 Design Constraints

1. System must follow airline operational rules
2. Unique IDs for flights, passengers, bookings
3. Role-based access control
4. Web-based application
5. Secure payment integration
6. Three-tier architecture
7. Database integrity maintained

3.6 Software System Attributes

3.6.1 Reliability

- Data consistency ensured

- Backup and recovery available

3.6.2 Availability

- Available 24/7
- Downtime only during maintenance

3.6.3 Security

- Password encryption
- Role-based access
- Secure sessions
- Limited login attempts

3.6.4 Maintainability

- Modular system design
- Easy updates and debugging

3.6.5 Portability

- Works on all browsers
- Deployable on Windows/Linux

4. SUPPORTING INFORMATION

4.1 Appendix A – System Workflow Summary

Summarised roles and responsibilities of each actor:

Actor	Responsibility
Admin	Manages flights; creates schedules; manages staff accounts; monitors bookings; generates reports.
Staff	Handles passenger check-in; assigns seats; updates flight status; assists boarding process.

Actor	Responsibility
Passenger	Searches flights; books tickets; makes payments; cancels bookings; views travel details and booking history.

4.2 Appendix B – Technology Stack

Core technologies used to build and run the system:

Layer	Technology
Frontend	HTML5, CSS3, JavaScript, Bootstrap
Backend	Java / PHP / Node.js with Express.js
Database	MySQL / PostgreSQL
Architecture	Three-tier: Presentation / Application / Database
Dev Tools	VS Code, WAMP Server, MySQL Workbench, Git & GitHub

---

4.3 Appendix C – Additional Notes (Optional but useful)

- The system ensures **secure ticket booking and payment processing**
- Role-based dashboards improve usability for Admin, Staff, and Passenger
- Designed for scalability to support future airline features

---

— End of Software Requirements Specification —